

Functional Programming 2026 — F# Study Notes

Sebastian Holt Svendsen

2026-05-27

Contents

Foundations (Lectures 1–2)	3
1. FP mindset	3
2. The REPL	3
3. Expressions	3
4. Immutability	4
5. Operators	4
6. Type inference & polymorphism	5
7. Functions	5
8. Pipelines & composition	6
Data Types (Lectures 1–2, 4)	7
1. Primitive & unit types	7
2. Tuples — anonymous grouping	7
3. Records — named fields	7
4. Discriminated Unions (algebraic data types)	8
5. Option & Result — the canonical DUs	9
6. Recursive types — trees & expressions	9
7. Lists — built-in recursive DU	10
8. Generic types	10
9. Exceptions (when a value can't express the error)	10
10. Record vs DU — which one?	11
Functions & Recursion (Lectures 1–5, 8)	12
1. Higher-order functions	12
2. map, filter, fold	12
3. Recursion	13
4. Mutual recursion	14
5. Recursion over recursive types	14
6. Tail recursion	14
Collections (Lectures 5, 6, 9)	17
1. Lists - immutable, singly-linked	17
2. Arrays - mutable, fixed-size, $O(1)$ indexed	17
3. Sets - immutable, unique, sorted	18
4. Maps - immutable key $\rightarrow$ value	18
5. Sequences - lazy, possibly infinite	18
6. Comprehensions	19
7. Common operations (work on every collection)	19
8. Conversions cheat sheet	20
9. List of $T \rightarrow$ set / seq / array of something <i>else</i>	20
Advanced FP (Lectures 10–12)	22
1. Monads	22

2. Parsers	23
3. Async, Concurrency & Parallelism	25
4. Binary & unary operators in a parser	28

Foundations (Lectures 1–2)

Cheat sheet for F# basics: the FP mindset, the REPL, expressions, immutability, operators, type inference, functions, and pipelines/composition.

Covered elsewhere: lists/records/tuples/DUs → 01-data-types; recursion + higher-order functions (map/filter/fold) → 02-functions-and-recursion.

1. FP mindset

- **Declarative**: describe *what* a result is, not *how* to compute it. Imperative/OO describe *how* (state + state-changing steps).
 - A program is a **mathematical function**; running it = **evaluating** it.
 - Built on the **λ -calculus** (Church, ~1930): $f(x) = x + 2$ is $\lambda x. x + 2$.
 - Core ideas: first-class & higher-order functions, type inference & polymorphism, recursion, algebraic data types, immutability.
-

2. The REPL

Input ends with `;;`. F# replies with `val <name> : <type> = <value>`.

```
> 2 * 3 + 4;;  
val it : int = 10           // `it` always names the last unnamed result
```

- A **binding** maps a name to a value: `price` $\mapsto$ 125.
- An **environment** is a collection of bindings.

```
> let price = 25 * 5;;           // val price : int = 125  
> let newPrice = price * 2;;    // val newPrice : int = 250  
// environment: [price |-> 125; newPrice |-> 250]
```

Primitive types: `int` (42), `float` (1.0), `bool` (`true`), `string` ("hi"), `char` ('a').

3. Expressions

In F# (almost) **everything is an expression** — it evaluates to a value. This replaces imperative *statements*, which only produce side effects.

If-expression — both branches required and must have the *same type*:

```
if b then e1 else e2  
1 + (if 1 > 1 then 1 else 2)    // = 3
```

`let ... in` (nested/local bindings): the local name is only in scope in the body.

```
let area =  
  let pi = 3.14159  
  let r = 2.0  
  pi * r * r
```

Evaluation = reduction by substitution ($\leadsto$ = “reduces to”):

```
fact 3 ~> 3 * fact 2 ~> 3 * 2 * fact 1 ~> ... ~> 3 * 2 * 1 * 1 ~> 6
```

4. Immutability

- Bindings are **immutable by default**: `let x = 5` cannot be reassigned.
- A second `let x = ...` **shadows** the old one (new binding, no mutation).
- Opt into mutation explicitly (used *sparingly*):

```
let mutable x = 0  
x <- x + 1           // `<-` assigns to a mutable binding
```

Why it matters: easier to reason about, no hidden side effects, safe to parallelise.

Side effects & sequencing (;)

A **side effect** is anything a function does besides return a value — printing, or mutating (`<-`, `arr.[i] <- x`). Such expressions return **unit** (`()`).

The semicolon **sequences** two expressions: `e1 ; e2` evaluates `e1` (for its effect, discarding its result), then evaluates `e2` and returns *its* value. The left side **must be unit**, else you get a warning.

```
let eat t p =  
  getLeftFork t p ; getRightFork t p    // run left (unit), then right; result = right
```

A **newline at the same indentation does the same thing** — usually preferred:

```
let eat t p =  
  getLeftFork t p           // unit  
  getRightFork t p          // value of the block = this last expression
```

- To discard a non-unit result so it can sit on the left, pipe it to **ignore**: `expensive () |> ignore`.
- Don’t confuse this `;` with the **separator** `;` in `[1; 2; 3]` or `{ A = 1; B = 2 }` — same symbol, different role.

5. Operators

Kind	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (<code>/</code> is integer division on <code>int</code>)
Comparison	<code>=</code> (equal), <code><></code> (not equal), <code><</code> <code>></code> <code><=</code> <code>>=</code>
Boolean	<code>&&</code> <code> </code> <code>not</code>
String	<code>+</code> (concatenation)

- **No implicit numeric conversion**: can’t mix `int` and `float`; convert with `float x / int x`.

- **Operators are functions.** Wrap in parentheses to use prefix:

```
(+) 2 3           // = 5
List.reduce (+) [1;2;3]
```

6. Type inference & polymorphism

F# **infers** types automatically (no annotations needed):

```
let rec pow = function
  | (x, 0) -> 1.0
  | (x, n) -> x * pow (x, n - 1)
// 1.0 is float => result float => x float => pow : float * int -> float
```

- When a type isn't constrained, F# **generalises** it to a **polymorphic** type using type variables 'a, 'b, ...

```
let id x = x           // id : 'a -> 'a
List.length           // 'a list -> int
```

- Annotate when you want to fix a type: `let f (x: int) : int = x + 1.`

7. Functions

Declaration mirrors value bindings; type is `<arg> -> <return>`:

```
let circleArea r = System.Math.PI * r * r // float -> float
```

Anonymous functions (`fun`) and **pattern functions** (`function`):

```
fun r -> System.Math.PI * r * r           // float -> float
function | 0 -> true | _ -> false         // int -> bool (`_` = wildcard)
```

Incomplete patterns compile but warn — cover all cases (use `_`).

Recursion needs the `rec` keyword:

```
let rec fact = function
  | 0 -> 1
  | x -> x * fact (x - 1)
```

Currying & partial application

`let f x y = ...` takes args **one at a time**; `let f (x, y) = ...` takes **one tuple**:

```
let f x y = x + y           // int -> int -> int    (curried)
let g (x, y) = x + y        // int * int -> int     (one pair)

f 5                         // ~> fun y -> 5 + y     (partial application)
let add5 = f 5              // specialise for later use
```

Prefer curried functions — partial application + first-class functions is the power combo.

8. Pipelines & composition

Pipe feeds a *value* into a function (reads left→right, fewer parens):

```
x |> f           // = f x
[1;2;3] |> List.map (fun x -> x * 2) |> List.sum // = 12
f <| x          // = f x   (reverse pipe, occasionally useful)
```

Composition combines *functions* into a new function:

```
let double x = x * 2
let inc     x = x + 1
let f = double >> inc      // (f >> g) x = g (f x): double first, then inc
let h = double << inc      // (f << g) x = f (g x): inc first, then double
```

Pipes work on values, composition works on functions. Both encourage step-by-step, transformation-based code.

Data Types (Lectures 1–2, 4)

How to model data in F#: primitives, tuples, records, discriminated unions (algebraic data types), `Option/Result`, and recursive types — plus pattern matching, the tool that takes them apart.

Covered elsewhere: immutability, type inference & polymorphism → 00-foundations; list/collection *operations* (`map/filter/fold`) → 02-functions-and-recursion and 04-collections.

1. Primitive & unit types

```
let i : int    = 42
let f : float  = 3.14
let b : bool   = true
let s : string = "hello"
let c : char   = 'a'
let u : unit   = ()    // "nothing" -- like void, but a real value
```

Annotations are optional — F# **infers** types. `int` and `float` never mix implicitly; convert with `float x / int x`.

2. Tuples — anonymous grouping

Fixed-size, ordered, **no field names**. Type is written with `*`.

```
let point = (3, 4)           // int * int
let triple = ("Alice", 30, true) // string * int * bool

let (x, y)    = point        // destructure: x = 3, y = 4
let (_, b, _) = triple       // wildcard _ ignores a component
fst (1, "a")    // 1         (pairs only)
snd (1, "a")    // "a"
```

Use **when**: a quick, throwaway grouping with no semantic weight.

3. Records — named fields

A fixed set of named fields that always coexist. Declare the type first.

```
type Person = { Name: string; Age: int }
```



```

let alice = { Name = "Alice"; Age = 30 } // field order doesn't matter
let n      = alice.Name                  // "Alice"
let older  = { alice with Age = 31 }     // copy-and-update; alice unchanged

// pattern match on fields
let greet { Name = n; Age = a } = printfn "%s is %d" n a

alice = { Age = 30; Name = "Alice" }    // true -- equality is structural

```

Use **when**: an entity has a fixed set of fields all present at once (Person, Config, Point). A record says “all of these together.”

4. Discriminated Unions (algebraic data types)

A value that is **exactly one** of several cases, each optionally carrying data. Also called *inductively defined types*, *tagged values*, *sum types*, or *disjoint unions*. Declared with **type** (not **let**); a DU concisely says **how the members of a type are created**.

```

type Shape =
  | Circle    of float
  | Square    of float
  | Triangle  of float * float * float

let c = Circle 1.2 // constructors are functions: Shape
let t = Triangle (3.0, 4.0, 5.0)

```

Cases **without data** behave like an enum (Java **enum**):

```

type Direction = North | South | East | West

```

Pattern matching

The way you consume a DU. The compiler **checks all cases are covered** (incomplete matches warn). A constructor only matches itself.

```

let area shape =
  match shape with
  | Circle r      -> System.Math.PI * r * r
  | Square a      -> a * a
  | Triangle (a,b,c) -> let s = (a + b + c) / 2.0
                        sqrt (s*(s-a)*(s-b)*(s-c))

```

Use **when**: a value is one of several mutually exclusive alternatives — a state machine, a command, success/failure. A DU says “*exactly one of these*.”

Unwrapping: binding through a pattern

A **let** doesn’t only bind names — it can bind through a **pattern**. For a **single-case** DU (a wrapper), that lets you pull the contents out in one line:

```

type Board = T of int[] // a single-case DU wrapping an array
let t = T [| 1; 2; 3 |]

let (T arr) = t // arr : int[] -- unwrapped in the binding
let area (T arr) = Array.sum arr // same trick in a function parameter

```

This is safe because the pattern is **irrefutable** — there’s only one case, so it always matches. (It’s exactly the `let (SM a) = ...` idiom in the monads section of 05-advancedFP.) The same works for any irrefutable pattern — tuples and records:

```
let (x, y)      = (3, 4)           // tuple
let { Name = n } = { Name = "A"; Age = 1 } // record field
```

! A **refutable** (multi-case) pattern compiles but warns and may crash at runtime if it doesn’t match:
`let (Some x) = o` or `let x :: rest = xs`. For those, use `match` instead.

...and “wrapper” types that aren’t single-case DUs? The analog differs: - `Async<'T>` — unwrap with `let!` `x = ...` inside an `async { }` workflow (→ 05-advancedFP). - `seq` / `Set` / `Map` — *abstract* types with no visible constructor, so there’s nothing to pattern-match; read them with module functions (`Seq.head`, `Set.minElement`, `Map.tryFind`) or convert (→ 04-collections).

5. Option & Result — the canonical DUs

`Option` replaces `null`: there either is a value or there isn’t.

```
// type 'a option = Some of 'a / None
let safeDivide a b = if b = 0 then None else Some (a / b)

match safeDivide 10 2 with
| Some v -> printfn "Result: %d" v
| None   -> printfn "Division by zero"
```

`Result` carries an error instead of just absence — errors without exceptions.

```
// type Result<'a,'e> = Ok of 'a / Error of 'e
let parseInt (s: string) : Result<int, string> =
    match System.Int32.TryParse s with
    | true,  v -> Ok v
    | false, _ -> Error $"{s}' is not a number"
```

6. Recursive types — trees & expressions

A type whose cases refer to itself: how you model hierarchical data.

```
type Tree =
    | Leaf
    | Node of Tree * int * Tree

let rec sum t =
    match t with
    | Leaf          -> 0
    | Node (l,n,r)  -> sum l + n + sum r
```

The classic **expression tree** (a tiny calculator) — data *represents* a program:

```
type Expr =
    | Num of int
    | Add of Expr * Expr
    | Mul of Expr * Expr
```

```
let rec eval = function
  | Num n      -> n
  | Add (e1, e2) -> eval e1 + eval e2
  | Mul (e1, e2) -> eval e1 * eval e2

eval (Mul (Add (Num 2, Num 3), Num 4)) // (2 + 3) * 4 = 20
```

Use when: data is inherently hierarchical or self-referential — trees, expressions, parsers. (Recurring over these → 02-functions-and-recursion.)

7. Lists — built-in recursive DU

Immutable, singly-linked, all elements the same type. Conceptually `[] | head :: tail`, so they pattern-match like any DU.

```
let xs = [1; 2; 3]
let ys = 0 :: xs      // prepend -> [0; 1; 2; 3] (cheap)
let zs = xs @ [4; 5]  // append  -> [1; 2; 3; 4; 5]

match xs with
| []      -> "empty"
| [x]     -> "one element"
| x :: rest -> $"head={x}, {List.length rest} more"
```

Operations (map/filter/fold) and other collections (arrays, sets, maps, sequences) live in 04-collections.

8. Generic types

Records and DUs can take type parameters ('a, 'b, ...):

```
type 'a Store = { Data: 'a; Owner: int option }
type 'a Tree  = Leaf | Node of 'a * 'a Tree * 'a Tree // tree of any element type
```

9. Exceptions (when a value can't express the error)

```
exception MyError of string
raise (MyError "went wrong")
failwith "quick error" // shorthand

try someRiskyOp ()
with
| MyError msg -> printfn "Error: %s" msg
| :? System.DivideByZeroException -> printfn "div by zero"
```

Prefer Option/Result for *expected* failures; reserve exceptions for the truly exceptional.

10. Record vs DU — which one?

Situation	Use
Fixed set of fields that always coexist	Record
Value is <i>one of</i> several distinct things	DU
State machine / game state	DU
Success / failure / absence	DU (Result , Option)
Hierarchical or recursive data	DU (recursive)
Config, entity, DTO	Record

```

type Player = { Name: string; Score: int }           // [x] always BOTH fields
type GameState =                                     // [x] exactly ONE at a time
  | Running of Player
  | Won      of Player
  | Draw

```

Quick comparison

Type	Named fields	Multiple cases	Recursive	Immutable
Tuple	✗	✗	✗	✓
Record	✓	✗	✗	✓ (default)
DU	per-case	✓	✓	✓
List	✗	✗	✓	✓

Functions & Recursion (Lectures 1–5, 8)

The two engines of F#: **higher-order functions** (functions that take or return functions - `map/filter/fold`) replace most loops, and **recursion** is how functions repeat. Then how to make recursion safe with **tail recursion**.

Covered elsewhere: function declaration, currying, partial application, pipes (`|>`) & composition (`>>`) → 00-foundations; the data types you recurse over (DUs, trees) → 01-data-types; collection-specific helpers → 04-collections.

1. Higher-order functions

A **higher-order function (HOF)** takes a function as an argument and/or returns one. Because functions are values in F#, this is ordinary. HOFs let you write short, declarative code and **replace loops** - done well, programs become concise and easy to read.

```
let twice f x = f (f x)           // ('a -> 'a) -> 'a -> 'a
twice (fun x -> x + 3) 10         // 16

let adder n = fun x -> x + n      // returns a function
let add5 = adder 5               // add5 7 = 12
```

2. map, filter, fold

Three HOFs cover most list processing. (They exist for every collection, e.g. `List.map`, `Array.map`, `Seq.map`.)

Function	Does	Signature (List)
<code>map</code>	transform every element	<code>('a -> 'b) -> 'a list -> 'b list</code>
<code>filter</code>	keep elements matching a predicate	<code>('a -> bool) -> 'a list -> 'a list</code>
<code>fold</code>	collapse a list to one value	<code>('s -> 'a -> 's) -> 's -> 'a list -> 's</code>

```
List.map    (fun x -> x * 2)    [1;2;3] // [2; 4; 6]
List.filter (fun x -> x % 2 = 1) [1;2;3;4] // [1; 3]
List.fold   (fun acc x -> acc + x) 0 [1;2;3] // 6
```

fold is the general loop. It threads an accumulator left-to-right:

```
fold f acc [x1; x2; ...; xn] = f (... (f (f acc x1) x2) ...) xn
```

Evaluation of `List.fold (-) 0 [1; 2; 3]` (note: left-associative):

```
fold (-) 0 [1;2;3] ~> fold (-) (0-1) [2;3] ~> fold (-) (-1-2) [3]
                  ~> fold (-) (-3-3) [] ~> -6
```

`foldBack` is the right-to-left twin (list first, state last): `('a -> 's -> 's) -> 'a list -> 's -> 's`.

```
List.foldBack (fun x acc -> x :: acc) [1;2;3] [] // [1; 2; 3] (rebuilds)
```

Chaining them

Compose a pipeline with `|>`. “Last digit of every odd number, as a string”:

```
let mergeLastDigits lst =
  lst
  |> List.filter (fun x -> x % 2 = 1) // drop even numbers
  |> List.map (fun x -> x % 10) // keep the last digit
  |> List.fold (fun acc x -> acc + string x) ""

mergeLastDigits [123; 124; 5; 1233] // "353"
```

The same thing point-free with composition (`>>`) reads worse - prefer the piped form. **Become good friends with `|>`, `>>`, `map`, `filter`, `fold`.**

3. Recursion

A function is **recursive** when it calls itself. F# requires the `rec` keyword, and you usually branch with pattern matching: a **base case** stops, a **recursive case** shrinks the problem toward it.

```
let rec fact (x: int) : int =
  match x with
  | 0 -> 1 // base case
  | x -> x * fact (x - 1) // recursive case
```

Evaluation = reduction (HR §1.4)

Substitute the definition repeatedly ($\rightsquigarrow$ = “reduces to”):

```
fact 3 ~> 3 * fact 2
      ~> 3 * (2 * fact 1)
      ~> 3 * (2 * (1 * fact 0))
      ~> 3 * (2 * (1 * 1))
      ~> 6
```

What happens in the stack and heap

Each call pushes a **stack frame** holding its parameters and the *work still pending after the recursive call returns* - here the multiplication `x * _`. The frames pile up to a depth equal to the recursion depth, then the base case fires and they **unwind**, doing the deferred work on the way out:

```
push fact 3 -> push fact 2 -> push fact 1 -> push fact 0 (stack grows)
return 1 -> 1*1 -> 2*1 -> 3*2 = 6 (stack unwinds)
```

The **stack** holds these frames; immutable values like list cells and closures live on the **heap**. Frames are finite, so recursion that is too deep (e.g. `fact 1_000_000`) overflows the stack with a `StackOverflowException`. The fix is tail recursion (§6).

4. Mutual recursion

Two functions that call each other are defined together with `let rec ... and ...`, so each is in scope for the other.

```
let rec isEven n = if n = 0 then true  else isOdd  (n - 1)
and      isOdd  n = if n = 0 then false else isEven (n - 1)

isEven 10  // true
```

5. Recursion over recursive types

Recursive *types* (01-data-types) are consumed by recursive *functions* with the same shape - one match case per constructor:

```
type Tree = Leaf | Node of Tree * int * Tree

let rec sum t =
  match t with
  | Leaf      -> 0 // base case
  | Node (l,n,r) -> sum l + n + sum r // recurse into both subtrees
```

6. Tail recursion

A call is in **tail position** if nothing is left to do after it returns. A function is **tail recursive** when its recursive call is in tail position - the result of the recursive call *is* the result. The compiler then reuses one stack frame instead of pushing new ones, so it **cannot overflow the stack** and runs like a loop.

A function is **not** tail recursive if, after the recursive call returns, the function still has work to do (e.g. `x * fact (x-1)` - the `*` is pending).

Two techniques turn a non-tail function into a tail-recursive one. The core difference is **what is carried along**: an accumulator stores *data*, a continuation stores *instructions*.

Accumulators - carry the result so far

Pass the partial result down as an extra argument; at the base case it's already complete, so there's nothing pending.

```
let sum xs =
  let rec loop acc = function
    | []      -> acc // answer already built
    | x :: xs -> loop (acc + x) xs // tail call - add as we go
  loop 0 xs

let reverse xs =
  let rec loop acc = function
    | []      -> acc
```

```

    | x :: xs -> loop (x :: acc) xs // builds the reversed list directly
loop [] xs

```

Continuations - carry the work to do later

Pass a function representing “what to do with the result.” You don’t compute now; you describe what should happen later, and the call stays in tail position.

```

let fact n =
  let rec loop n cont =
    match n with
    | 0 -> cont 1 // run the accumulated plan
    | n -> loop (n - 1) (fun r -> cont (n * r))
  loop n id

```

cont = “what to do with the result later”; each step extends the plan and the recursive call is the last thing done.

Tail recursion, accumulators, and continuations: slides **Lecture 8** (optimisation), or HR chapters **8.1–8.6** and **9**.

MATCH WITH vs FUNCTION

function is shorthand for an anonymous function that immediately pattern matches on its single argument.

```

let f =
  function
  | [] -> 0
  | x::xs -> x

```

is equivalent to:

```

let f xs =
  match xs with
  | [] -> 0
  | x::xs -> x

```

key constraint: function only works when you are defining a function with exactly one implicit argument. it introduces that argument automatically.

once you already have named parameters, function becomes wrong:

```

let g xs cont =
  function
  | [] -> cont 0
  | x::xs -> cont x

```

this is actually:

```

let g xs cont =
  fun arg ->
    match arg with
    | [] -> cont 0
    | x::xs -> cont x

```

so g now takes 3 arguments, not 2.

correct version:

```

let g xs cont =
  match xs with

```



```
| [] -> cont 0  
| x::xs -> cont x
```

Collections (Lectures 5, 6, 9)

F#'s five workhorse collections - **lists**, **arrays**, **sets**, **maps**, **sequences** - plus **comprehensions** and the conversions between them. All except arrays are immutable by default.

Covered elsewhere: the list type as a recursive DU → 01-data-types; the higher-order functions (map/filter/fold) these all share → 02-functions-and-recursion.

Collection	Literal	Mutable?	Ordered?	Lookup	Best at
List	[1; 2; 3]	no	yes	O(n)	recursion, prepend
Array	[1; 2; 3]	yes	yes	O(1) index	random access, mutation
Set	set [1; 2; 3]	no	sorted	O(log n)	membership, uniqueness
Map	Map.ofList [(1,"a")]	no	sorted by key	O(log n)	key → value lookup
Seq	seq { 1; 2; 3 }	no	yes	lazy	large / lazy / infinite

1. Lists - immutable, singly-linked

All elements one type. Cheap to prepend (::), O(n) to index or append.

```
let xs = [1; 2; 3]
let ys = 0 :: xs      // prepend -> [0; 1; 2; 3]    (O(1))
let zs = xs @ [4; 5]  // append -> [1; 2; 3; 4; 5] (O(n))
xs.[1]                // 2    (indexing is O(n) - avoid in loops)
[1 .. 5]              // [1; 2; 3; 4; 5]    range
[1 .. 2 .. 9]         // [1; 3; 5; 7; 9]    range with step

match xs with
| []      -> "empty"
| x :: rest -> $"head {x}, {List.length rest} more"
```

Use **when**: you process front-to-back / recursively - the default FP collection.

2. Arrays - mutable, fixed-size, O(1) indexed

Written [| ... |]. The one **mutable** built-in collection: update in place.

```

let arr = [| 10; 20; 30 |]
arr.[0] // 10 - get (O(1))
arr.[0] <- 99 // mutate index 0 in place
arr.Length // 3

// creating arrays
Array.create 3 0 // [| 0; 0; 0 |]
Array.init 4 (fun i -> i * i) // [| 0; 1; 4; 9 |]
Array.zeroCreate 3 // [| 0; 0; 0 |]
[| 1 .. 5 |] // [| 1; 2; 3; 4; 5 |]

```

Use **when**: you need fast random access or genuine in-place mutation (e.g. performance-sensitive numeric code).

3. Sets - immutable, unique, sorted

An unordered collection of **distinct** values (kept sorted internally). Adding a duplicate is a no-op. Operations return a *new* set.

```

let s = set [3; 1; 2; 2] // {1; 2; 3} - duplicate dropped
let s2 = Set.add 4 s // {1; 2; 3; 4}
Set.contains 2 s // true (O(log n))
Set.union (set [1;2]) (set [2;3]) // {1; 2; 3}
Set.intersect (set [1;2]) (set [2;3]) // {2}
Set.difference (set [1;2;3]) (set [2]) // {1; 3}

```

Use **when**: you care about membership and uniqueness, not order or position.

4. Maps - immutable key → value

An immutable dictionary, sorted by key. Keys are unique.

```

let m = Map.ofList [(1, "one"); (2, "two")]
let m2 = Map.add 3 "three" m // returns a new map
m.[1] // "one" (throws if missing)
Map.tryFind 2 m // Some "two" (safe - returns option)
Map.tryFind 9 m // None
Map.containsKey 2 m // true
Map.remove 1 m // map without key 1

// Idiomatic safe lookup with a default
match Map.tryFind k m with
| Some v -> v
| None -> "missing"

```

Use **when**: you look things up by key - counts, lookups, caches.

5. Sequences - lazy, possibly infinite

seq is F#'s lazy collection (IEnumerable<'T>): elements are **computed on demand**, so a sequence can be infinite as long as you only take a finite prefix.

```

let nums = seq { 1; 2; 3 }
let sq    = seq { for i in 1 .. 5 -> i * i }    // lazy - nothing computed yet

Seq.take 3 sq |> Seq.toList                      // [1; 4; 9]    forces 3 elements

// infinite sequence - fine because it's lazy
let naturals = Seq.initInfinite (fun i -> i)    // 0, 1, 2, 3, ...
naturals |> Seq.take 5 |> Seq.toList             // [0; 1; 2; 3; 4]

```

Use **when**: data is large, streamed, generated, or infinite, and you only need part of it. (Lists/arrays are *eager* - fully built immediately.)

6. Comprehensions

A **comprehension** builds a collection by describing its elements with a `for` loop - `-> expr` yields one element per item, `do ... yield` gives full control. The bracket type chooses the collection: `[ ]` list, `[| |]` array, `seq { }` sequence (wrap in `set/Map.ofList` for those).

```

[ for x in 1 .. 5 -> x * x ]                    // [1; 4; 9; 16; 25]    (list)
[| for x in 1 .. 5 -> x * x |]                  // array version
seq { for x in 1 .. 5 -> x * x }                // lazy version

// filter with `do ... if ... then yield`
[ for x in 1 .. 10 do if x % 2 = 0 then yield x ] // [2; 4; 6; 8; 10]

// multiple generators (nested loops)
[ for x in 1 .. 2 do for y in 1 .. 2 -> (x, y) ] // [(1,1);(1,2);(2,1);(2,2)]

set [ for x in [1;2;3] -> x ]                   // {1; 2; 3}

```

7. Common operations (work on every collection)

The same higher-order functions exist per module - `List.`, `Array.`, `Set.`, `Map.`, `Seq.`:

```

List.map      (fun x -> x * 2) [1;2;3]          // [2; 4; 6]
List.filter   (fun x -> x > 1) [1;2;3]          // [2; 3]
List.fold     (fun acc x -> acc + x) 0 [1;2;3]  // 6
List.sum      [1;2;3]                          // 6
List.length   [1;2;3]                          // 3
List.rev      [1;2;3]                          // [3; 2; 1]
List.sortBy   abs [-3; 1; -2]                  // [1; -2; -3]
List.contains 2 [1;2;3]                        // true

[1;2;3;4]
|> List.filter (fun x -> x % 2 = 0)
|> List.map (fun x -> x * 10)
|> List.sum                                     // 60

```

8. Conversions cheat sheet

```
// between collections
List.toArray lst      Array.toList arr
List.toSeq    lst      Seq.toList    s
Array.toSeq   arr      Seq.toArray   s
Set.ofList    lst      Set.toList    s
Map.ofList    pairs    Map.toList    m

// strings <-> collections
s.ToCharArray ()          // string -> char[]
System.String arr         // char[] -> string
System.String (List.toArray lst) // char list -> string (or String.Concat)
s |> Seq.toList           // string -> char list

// splitting strings
s.Split(' ')              |> Array.toList      // by one char
s.Split([| ','; ';' |]) |> Array.toList      // by several chars

// chunk a string into pieces of length n
[ for i in 0 .. n .. s.Length - 1 -> s.Substring(i, min n (s.Length - i)) ]

// build a string from an int list via a pipeline
let lastDigitsString lst =
    lst |> List.map (fun x -> x % 10) |> List.map string |> String.concat ""
```

9. List of T → set / seq / array of something else

A very common task: given a `list`, produce a `Set` (or `seq`/`array`) of some **derived** element type. Always two steps - (1) **map** each element to its piece(s), then (2) **collect** the pieces into the target collection.

Each element → one value

`map`, then convert with the target module:

```
[1;2;3] |> List.map (fun x -> x * x) |> Set.ofList // set {1; 4; 9}
[1;2;3] |> List.map string              |> List.toArray // [| "1"; "2"; "3" |]
```

Each element → a *collection*, then merge

When `f x` itself returns a `set`/`list`, you must **combine** the results:

```
// list of sets -> ONE set (e.g. rectangles -> their union of coordinates)
rects |> List.map coordsAcc |> Set.unionMany // unions every set
// Set.unionMany : seq<Set<'a>> -> Set<'a>

xs |> List.collect f // map + flatten = List.map f >> List.concat
xs |> List.map f |> Array.concat // list of arrays -> one array
xs |> List.map f |> Seq.concat // list of seqs -> one seq
```

This is the fix for a “list → set” merge: each element already gives a `Set`, so combine with `Set.unionMany` (or `List.fold Set.union Set.empty`) - **not** `Set.add`, which adds a single *element*, not a whole set.

...and back again (reverse) - pick the *target* module

Conversions are pure and work in any direction:

from ↓ / to →	List	Array	Set	Seq
List	-	List.toArray	Set.ofList	List.toSeq
Array	Array.toList	-	Set.ofArray	Array.toSeq
Set	Set.toList	Set.toArray	-	Set.toSeq
Seq	Seq.toList	Seq.toArray	Set.ofSeq	-

Async / parallel version

When producing each piece is *async* work you want to overlap, map to Async<_>, run with Async.Parallel (which yields an **array**), then collect:

```
items
|> List.map (fun x -> async { return expensive x }) // Async<'b> list
|> Async.Parallel // Async<'b []>
|> Async.RunSynchronously // 'b []
|> Set.ofArray // collect -> set (or Array.toList,
↪ ...)
```

Only the **work** is async - the conversion itself is always pure/synchronous, so the “reverse” is just the normal Array.toList / Set.ofArray step at the end. (async / Async.Parallel details → 05-advancedFP.)

Advanced FP (Lectures 10–12)

Three advanced topics: **monads** (chaining computations that carry context), **parser combinators** (building big parsers from small ones), and **async / concurrency / parallelism**.

Covered elsewhere: **bind**, **Option/Result**, expression trees → 01-data-types; combinators are higher-order functions → 02-functions-and-recursion.

1. Monads

What is a Monad?

A monad is a wrapper around a value that also carries some extra context (like state, errors, lists, etc.). The wrapper lets you chain computations together without manually threading that context everywhere.

A State Monad specifically carries:

- A computation (a function)
- That computation takes a state as input and returns a (result, new state) pair

So a state monad wraps a function **state** → (**value**, **state**)

The core idea

A monad is something that has:

- a way to wrap a value
- a way to chain computations

In F# terms:

ret — **wrapping** (return / unit)

- Wraps a plain value into the monad, doing nothing to the state

```
let ret x = SM (fun s -> Some (x, s))
//
//           ^      ^
//           result state unchanged
```

ret 5 gives you a state computation that, given any state **s**, returns (5, **s**). It doesn't touch the state.

fail

```
let fail = SM (fun _ -> None)
```

just a computation that always fails. use it in **bind**.

bind

```
let bind f (SM a) : StateMonad<'b> =
    SM (fun s ->
        match a s with
        | Some (x, s') -> let (SM g) = f x
                           g s'
        | None -> None)
```

This chains two computations together. Step by step:

- runs the first computation `a` on the current state `s`
- If that succeeds with `(x, s')`, it calls `f x`. This gives back a new monadic computation (let's call it `g`)
- It then runs `g` on the updated state `s'`
- If step 1 fails, the whole thing returns `None`. short-circuit, no further computation runs

This is how state is threaded through a seq of steps without having to pass it manually.

`>>=` is just `bind` with arguments flipped:

```
let (>>=) x f = bind f x
```

So computation `>>= f` means “run this computation, then pass the result to `f`”. It lets you write pipelines left-to-right:

```
getChar >>= (fun c -> checkLetter c >>= (fun _ -> ret c))
```

`>>>=` is “sequence” — run two computations in order, but throw away the result of the first:

```
let (>>>=) x y = x >>= (fun _ -> y)
```

Useful when you only care about the side effect (state update) of the first computation.

Monad laws:

- Law 1: `ret a >>= f = f a` (“left identity”)

Wrapping `a` in `ret` and then binding `f` is the same as just calling `f a` directly. `ret` does nothing interesting.
- Law 2: `m >>= ret = m` (“right identity”)

Binding `ret` onto a computation does nothing. `ret` just re-wraps the value without touching state, so `m >>= ret` gives back the same result as `m`.
- Law 3: `(m >>= f) >>= g = m >>= (fun x -> f x >>= g)` (“associativity”)

Chaining doesn't care about parenthesisation. Whether you go `(m then f) then g` or `m then (f then g)`, you get the same result. This is what lets you build long pipelines without worrying about grouping.

2. Parsers

Parsers — basics

Parsing is the process of taking raw input (usually a string) and converting it into a structured form.

Examples:

```
"123"          -> integer 123
"a + b * c"     -> expression tree
"if x then y else z" -> AST node
```

A parser is basically a function:

- input string -> parsed value + remaining input (or failure)

Combinator parsers:

Instead of writing one big parser, you build small parsers and combine them. small parsers + combinators = big parser

Transformation (mapping)

|>> (map) — transforms parsed result

```
pint32 |>> (fun x -> x + 1)
```

Meaning: parse int, then add 1

Sequencing

.>>. — (AND THEN, keep both results)

run parser A then parser B, return tuple (A result, B result)

```
pchar '(' .>>. pint32 // Result: ('(', 123)
```

.>> keeps only left side, >>. keeps only right side.

OR

<|> — try left parser, if it fails try right parser

```
pchar '+' <|> pchar '-'
```

REPETITION

many p — parse 0 or more occurrences

many1 p — parse 1 or more occurrences

```
many (satisfy isLetter) // Result: list of chars
```

OPTIONAL PARSING

opt p — optional parser

Returns: Some value if found, None if not found

COMMON HELPERS

between pOpen pClose pContent — parses content inside delimiters

```
between (pchar '(') (pchar ')') pint32 // number inside parentheses
```

LIST PARSERS

sepBy p sep — parse list of p separated by sep

```
sepBy pint32 (pchar ',') // "1,2,3" -> [1; 2; 3]
```

RECURSIVE PARSERS

Sometimes grammar is recursive:

```
expression -> expression + term | term
```

To define this you use `createParserForwardedToRef`. This allows:

- defining parser before it exists
 - filling it later
-

3. Async, Concurrency & Parallelism

3.1 Intro

three related but different ideas:

- **Synchronous execution:** one thing happens at a time
- **Concurrency:** multiple tasks are *in progress* (interleaved)
- **Parallelism:** multiple tasks run *at the same time*

In F#, `async {}` is mainly about **concurrency**, not automatic parallelism.

3.2 What is `Async<'T>`?

An `Async<'T>` is:

a *description* of a computation that will eventually produce a value of type `'T`

It does NOT run immediately.

```
let work =  
    async {  
        return 1 + 1  
    }
```

At this point: nothing has executed — it's just a recipe.

3.3 How to RUN async code

`Async.RunSynchronously`

```
let result = Async.RunSynchronously work
```

- runs the async computation
- blocks the current thread
- returns `'T`

Think: “run async code like normal blocking code”.

`Async.Start`

```
Async.Start work
```

- runs in background
- does NOT block
- returns `unit`

You don't get the result directly.

3.4 Async workflows (async {})

```
async {  
    let! x = asyncComputation  
    return x + 1  
}
```

Key ideas:

- `let!` = wait for async result
- `return` = wrap result
- `return!` = chain async computations

3.5 Concurrency vs Parallelism

Concurrency (Async) — F# switches between tasks during waits.

```
Task A: ----wait----  
Task B: ----wait----
```

Parallelism (CPU cores) — actually simultaneous execution.

```
Core 1: Task A running  
Core 2: Task B running
```

3.6 Async.Parallel

```
Async.Parallel : Async<'T> list -> Async<'T []>
```

It means: run multiple async computations at the same time and collect results.

```
let tasks =  
    [  
        async { return 1 }  
        async { return 2 }  
        async { return 3 }  
    ]
```

```
Async.Parallel tasks  
|> Async.RunSynchronously  
// [1; 2; 3]
```

3.7 Common mistake (VERY IMPORTANT)

WRONG — inputs are NOT `Async<'T>`, they are plain values:

```
[1;2;3]  
|> List.map (fun x -> x + 1)  
|> Async.Parallel
```

RIGHT:

```
[1;2;3]  
|> List.map (fun x -> async { return x + 1 })  
|> Async.Parallel
```

3.8 Why Async.Parallel often “does nothing”

If your work is tiny (like string operations), CPU cheap, or has no waiting (I/O), then overhead > benefit — it may feel like nothing changed.

3.9 Async.RunSynchronously danger

```
Async.RunSynchronously (Async.Parallel tasks)
```

This blocks the main thread, waits for all tasks, and removes the async benefit at top level. Use mainly in scripts or the main entry point.

3.10 let! vs let

```
let x = asyncComputation // x is Async<'T> (no wait)
let! x = asyncComputation // x is 'T (waits)
```

3.11 return vs return!

```
return 5 // wrap value -> Async<int>
return! otherAsync // flatten async
```

3.12 Why a Caesar example fails

```
text.Split(' ')
|> Array.map (fun str -> encrypt str offset)
```

Problem: encrypt returns string, not Async<string>, so Async.Parallel cannot be used. Fix:

```
text.Split(' ')
|> Array.map (fun str -> async { return encrypt str offset })
|> Async.Parallel
```

3.13 When should you use Async?

Use Async when: file IO, network calls, waiting (sleep, delays), independent tasks that can overlap.

DON'T use it when: simple loops, small CPU work, pure transformations.

3.14 When should you use Parallel?

Use parallel when: CPU-heavy work, independent computations, large datasets.

```
Array.Parallel.map expensiveFunction data
```

3.15 Simple mental model

- Async = “I will wait for something later”
- Parallel = “I want multiple CPU cores now”

3.16 Summary

- Async<'T> = description of work
- Async.RunSynchronously = run and block
- Async.Start = run in background

- `Async.Parallel` = run multiple async tasks together
- `async` does NOT mean parallel automatically

4. Binary & unary operators in a parser

(Builds on §2 Parsers; examples from Assignment 10's expression parser and Lecture 11 "Parser Combinators".)

When you parse a language, operator **syntax** (`a + b`, `-a`, `x = y`) becomes **AST nodes** (`Add(a,b)`, `Mul(Num -1, a)`, `Eq(x,y)`). Operators come in two shapes, and the parser captures each with one tiny combinator:

- **binary operator (binop)** — an operator *between* two operands: `a + b`
- **unary operator (unop)** — an operator on *one* operand: `-a`, `~b`

Prerequisite: whitespace-eating combinators

Built from the base combinators `.>>.` (keep both), `.>>` (keep left), `>>.` (keep right) plus `spaces = many whitespaceChar`:

```
let (.>*>.) p1 p2 = (p1 .>> spaces) .>> p2 // run both, swallow spaces between
let (.>*>) p1 p2 = p1 .>*> p2 |>> fst // keep LEFT result
let (>*>.) p1 p2 = p1 .>*> p2 |>> snd // keep RIGHT result
```

The two combinators

```
let unop op a = op >*> a // parse op, throw it away, return a's result
// unop : Parser<'op> -> Parser<'a> -> Parser<'a>

let binop op a b = a .>*> op .>*> b // parse a, then op (discarded), then b
// binop : Parser<'op> -> Parser<'a> -> Parser<'b> -> Parser<'a * 'b>
```

- `unop` returns the **single** operand (the operator symbol is consumed but dropped).
- `binop` returns a **tuple** (`left`, `right`) — because a binary operator has two operands.

Turning the parse into an AST node with `|>>`

`|>>` maps the parser's result. A `binop` gives a tuple, which lines up exactly with a 2-argument constructor; a `unop` gives a single value for a 1-argument one:

```
let AddParse = binop (pchar '+') ProdParse TermParse |>> Add <?> "Add"
// (x, y) -> Add (x, y) because Add : aexpr * aexpr -> aexpr
let NotParse = unop (pchar '~') BoolArithParse |>> Not <?> "Not"
// b -> Not b
```

When the operator is **not** a primitive AST case, map with a lambda instead — desugaring it into the core constructors:

```
// subtraction: a - b == a + (b * -1)
let SubParse = binop (pchar '-') ProdParse TermParse
|>> (fun (x, y) -> Add (x, Mul (y, Num -1)))
// unary minus: -a == -1 * a
let NegParse = unop (pchar '-') AtomParse
|>> (fun x -> Mul (Num -1, x))
```

(`<?>` "name" just labels a parser for nicer error messages.)

Precedence = layering the grammar

A naive grammar like $A \rightarrow A + A \mid A * A \mid x$ is **ambiguous** and **left-recursive** (Lec 11). You fix both by splitting it into one *non-terminal per precedence level*, loosest-binding at the top:

```
Term  -> Prod + Term   | Prod          (+ , -   bind loosest)
Prod  -> Atom * Prod   | Atom           (* , /   bind tighter)
Atom  -> int | -Atom | ( Term )         (literals/parens bind tightest)
```

Each level is just `binop/unop + choice`, and the mutual recursion (an `Atom` can contain a whole `Term` inside parentheses) is tied together with `createParserForwardedToRef` — declare a parser before it's defined, fill it in later with `:=`:

```
let TermParse, tref = createParserForwardedToRef<aexpr>()
let ProdParse, pref = createParserForwardedToRef<aexpr>()
let AtomParse, aref = createParserForwardedToRef<aexpr>()

let AddParse = binop (pchar '+') ProdParse TermParse |>> Add
do tref := choice [AddParse; ProdParse]           // + : loosest

let MulParse = binop (pchar '*') AtomParse ProdParse |>> Mul
do pref := choice [MulParse; AtomParse]           // * : tighter

let NParse = pint32 |>> Num
let NegParse = unop (pchar '-') AtomParse |>> (fun x -> Mul (Num -1, x))
let ParParse = parenthesise TermParse             // ( ) re-enters the top
do aref := choice [NParse; NegParse; ParParse]    // atoms : tightest
```

Derive everything from a minimal core

Keep the AST small (`Add`, `Mul`, `Div`, `Eq`, `Lt`, `Conj`, `Not`) and define the rest as **smart constructors**; the operator parsers then just map onto those:

```
let (.-.) a b = Add (a, Mul (b, Num -1)) // subtraction
let (.||.) b1 b2 = ~(~b1 .&&. ~b2) // a\ / b = ~(~a /\ ~b) (De Morgan)
let (<.>.) a b = ~(a .<= b) // a<>b = ~(a=b)
let (<=.) a b = a .<. b .||. ~(a .<>. b) // a<=b = a<b \ / a=b
let (>=.) a b = ~(a .<. b) // a>=b = ~(a<b)
let (>.) a b = ~(a .<= b) .&&. (a .>= b) // a>b = a<>b /\ a>=b

let GtOrEqParse = binop (pstring ">=") TermParse A1Parse |>> (fun (a,b) -> a .>= b)
```

So you only write *real* parser/eval logic for a handful of operators; `>=`, `<=`, `<>`, `\ /`, `-`, `%` are all just rewrites.

One-line mental model

	shape	combinator	result	example map
Unary	op a	unop op a	the operand	unop (pchar '~') b >> Not
Binary	a op b	binop op a b	(a, b) tuple	binop (pchar '+') p t >> Add